

QR Code Following Case

QR Code Following Case

1. Course Content
2. Start Agent
3. Run Case
 - 3.1 Dynamic Parameter Adjustment
4. Source Code Analysis

1. Course Content

- Run example programs, the car will recognize detected QR codes and frame them, then the car will continuously follow the QR code to keep it in the center of the screen, and maintain a distance of 0.4 meters from the recognized QR code. Press the `q/Q` key to exit the program.

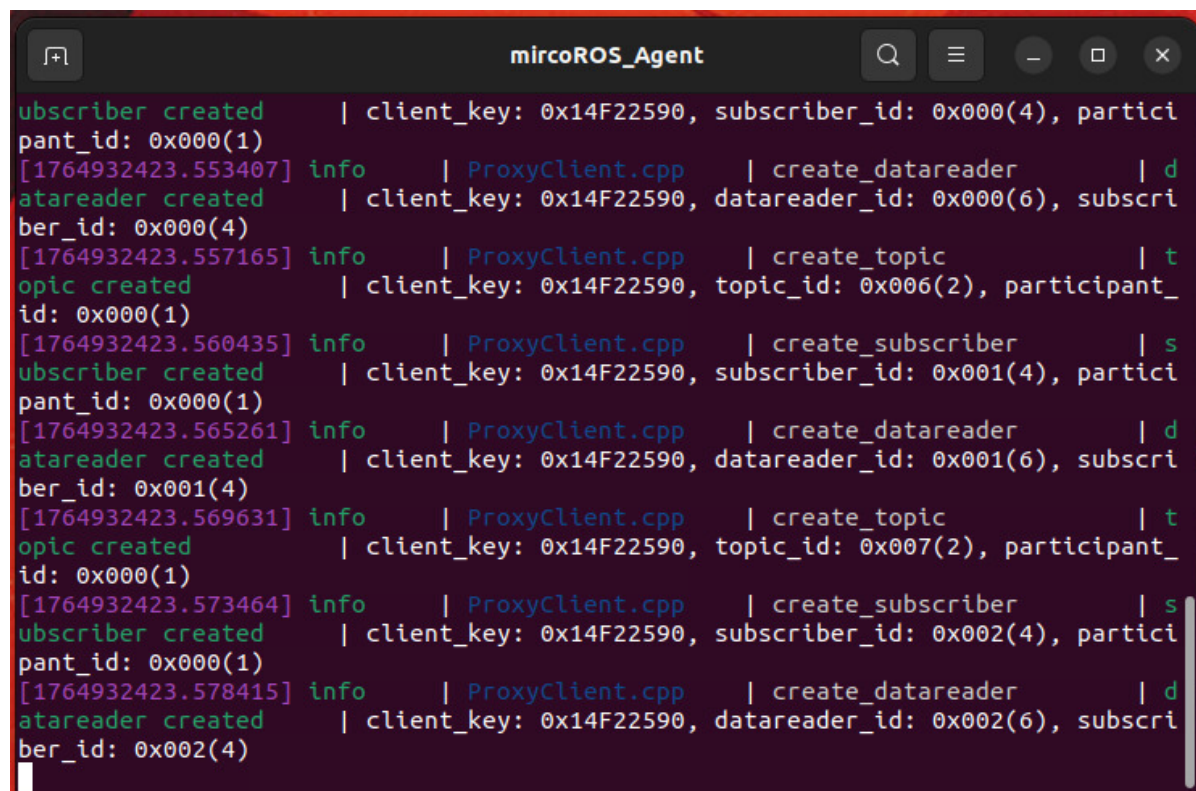
2. Start Agent

Note: If already started, no need to restart

Enter command in the car terminal:

```
start_agent
```

The terminal prints the following information, indicating successful connection



```
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1764932423.553407] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1764932423.557165] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1764932423.560435] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1764932423.565261] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1764932423.569631] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1764932423.573464] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1764932423.578415] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

3. Run Case

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

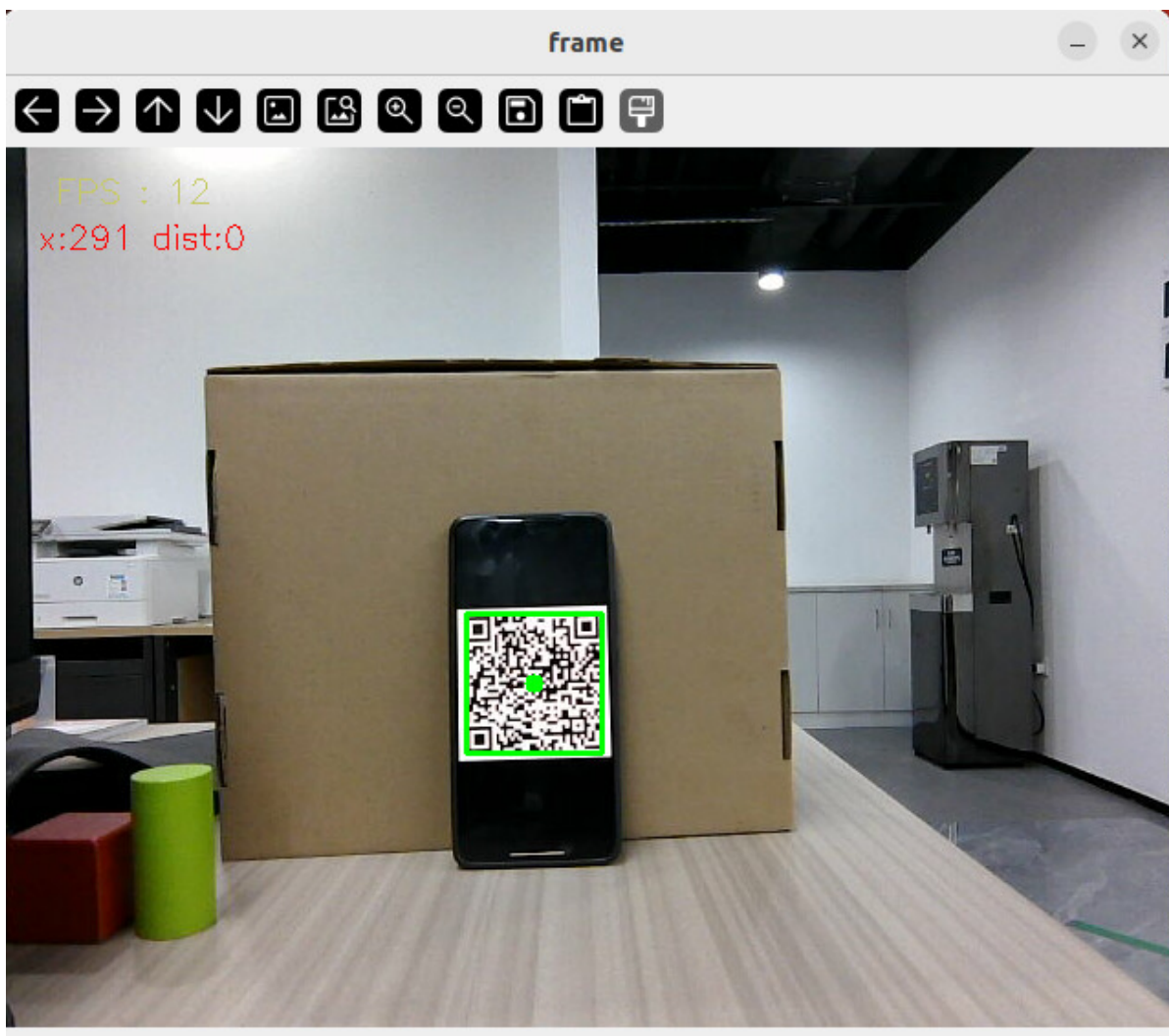
Open a terminal inside the container,

```
exec_m3
```

- Enter command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=qrFollow
```

- After the program starts, when a QR code is detected, it will automatically frame the QR code area, and the car will start following.

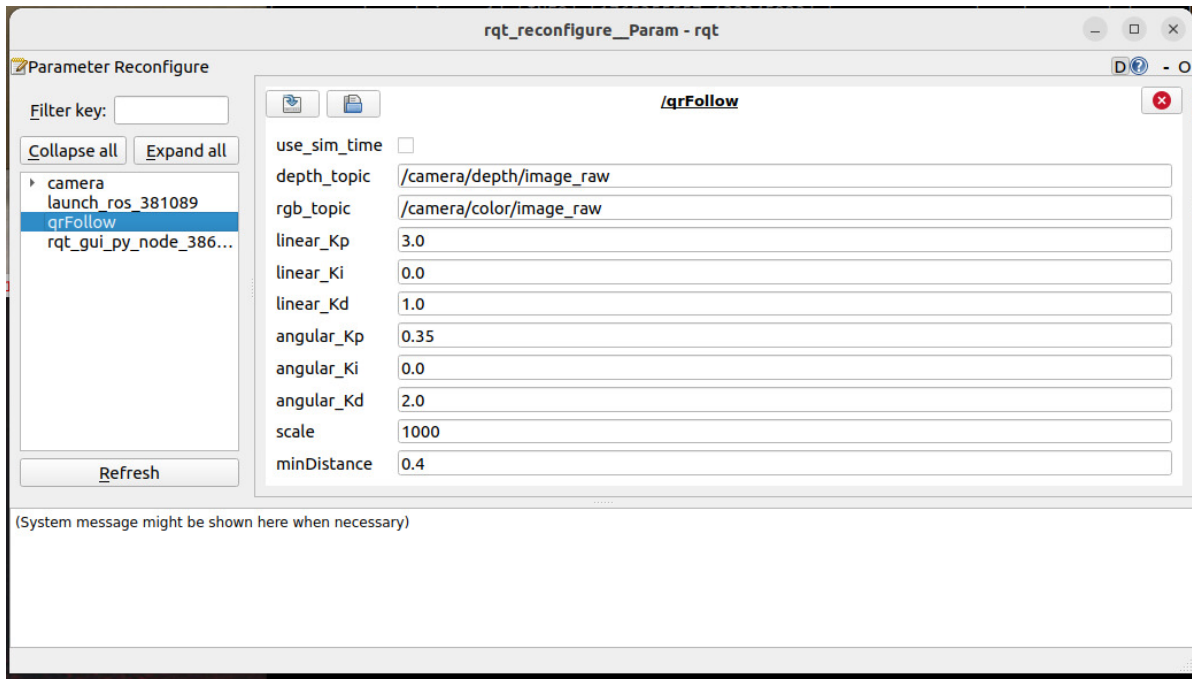


[!TIP]

If the car runs with poor stuttering effect, you can replace it with a QR code photo that can better obtain depth information. This is because if the QR code depth information is 0, the car will stop

3.1 Dynamic Parameter Adjustment

```
ros2 run rqt_reconfigure rqt_reconfigure
```



- After modifying parameters, click on the blank area of the GUI to write parameter values. Note that it will only take effect for the current startup. For permanent effect, you need to modify the parameters in the source code.

Parameter analysis:

【linear_Kp】 , 【linear_Ki】 , 【linear_Kd】 : Linear speed PID control during car following process.

【angular_Kp】 , 【angular_Ki】 , 【angular_Kd】 : Angular speed PID control during car following process.

【minDistance】 : Following distance, always maintain this distance.

4. Source Code Analysis

qrFollow.py source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/qrFollow.py
```

- This program mainly has the following functions:
- Subscribe to depth camera topics to get topic images;
- Use OpenCV's QR code detector to detect QR codes;
- Process images to get QR code screen center coordinates and depth distance information.
- Calculate distance PID and center coordinate PID to get car forward speed distance and turning angle.

Some core code is as follows,

```
#QR Code Detection and Tracking
processed_frame, (x, y, w, h) = detect_qrcode(result_image) # QR code detection
function
```

```

if processed_frame is not None:
    # Use the processed labeled image
    result_image = processed_frame

    # Calculate the center coordinates of the QR code
    self.Center_x = x + w // 2
    self.Center_y = y + h // 2
    self.Center_r = x * w // 100

    # Mark the center point
    cv.circle(result_image, (self.Center_x, self.Center_y), 5, (0,255,0), -1)

#Test target point distance
points = [
    (int(self.Center_y), int(self.Center_x)),      # Center Point
    (int(self.Center_y+1), int(self.Center_x+1)),  # Bottom right offset
    (int(self.Center_y-1), int(self.Center_x-1))   # Upper left offset
]
valid_depths = []
for y, x in points:
    depth_val = depth_image_info[y][x] # Get the depth value
    if depth_val != 0: # Filter invalid values
        valid_depths.append(depth_val)
# calculate the average distance
dist = int(sum(valid_depths)/len(valid_depths)) if valid_depths else 0

#Start the control thread
threading.Thread(target=self.execute).start()

# Calculate the movement speed and turning angle using the PID algorithm based on
the x and y values
def execute(self, rgb_img, action):
    #PID calculation deviation
    linear_x = self.linear_pid.compute(dist, self.minDist)
    angular_z = self.angular_pid.compute(point_x, 320)
    # The car is in the parking area and the speed is 0
    if abs(dist - self.minDist) < 30: linear_x = 0
    if abs(point_x - 320.0) < 30: angular_z = 0
    twist = Twist()
    ...
    # Publish the calculated speed information
    self.pub_cmdvel.publish(twist)

```

